MEMORANDUM

From:	William Waite wwaite@calpoly.edu
Course:	ME 4305-S03: Mechatronics II
Date:	October 28, 2026
RE:	**Prelab 0x08 – Individual Firmware Audit**

Latest Code Version

Our most up-to-date collection of Romi firmware is present in the Lab 0x08 subfolder of the team's shared OneDrive folder, titled "ME-4305-mecha26". The 15 Python scripts consist of the 4 drivers, 4 task classes, 2 auto plotters, 2 calibration files, closed loop class, parameter configuration file, and main execution file. Aside from the auto plotter scripts, these working files are all currently flashed to the Romi.

Application Screening

I respond to the provided screening questions in Table 1 below. I include written context for the responses where caveats and additional information are more useful.

Table 1. Screening questions and responses.

ID	Screening question	Response
A	Can you explain how startup, scheduling, and the current operating modes connect, and does the implementation agree with the task diagram?	**No**. The task diagram does not yet agree with the new implementation of the IMU task. Also, the scheduler performs with Line and IMU task latency (main, taskuser, taskmotor, taskline, taskimu).
B	Do task invocations perform bounded work and return or yield without waiting for an entire test, maneuver, or user interaction?	**Yes**, but the line sensor's ADC acquisition and averaging delay other tasks during its call. Task executions are bounded overall (taskmotor, taskline, taskimu, taskuser, linesensor).
C	Can sensing, control, user input, and data handling make progress while the robot is operating?	**Yes**, but with the timing limitation of the sensor specified above (main, taskmotor, taskline, taskimu, taskuser, linesensor).
D	Do task state transitions remain within their owning tasks, with other tasks communicating requests or data through understandable interfaces?	**Yes** (main, taskmotor, taskline, taskimu, taskuser).

ID	Screening question	Response
E	Are shared values' meanings, units, signs, and update responsibilities clear, including which component owns each hardware resource?	**Yes** (main, taskmotor, taskline, taskimu, taskuser).
F	Do the drivers and tasks handle the limits and failure cases relevant to their present use, including initialization and stopping?	**Partially**. Faults or stopping during preparation of the Line task will execute *return*, so the task will terminate before normal operation is complete (main, taskline, taskimu, closedloop, linesensor).
G	Can a local change be made without unrelated edits throughout the program, and can you identify the components worth retaining?	**Yes**. This is why our romi_config file of configuration settings is worth retaining (romi_config, main).
H	Can every current group member independently retrieve the same code and configuration, run it, and preserve changes?	**Yes** (Shared OneDrive folder, with the latest code in the Lab 0x08 subfolder).

Task Tracing and Concerns

Task traced: `TaskLine.run()` in taskline.py, during `S4_TURN_BACK`.

On resuming after the previous yield, the task begins another loop iteration and calls `_run_active_state(ticks_us())`. This checks motor faults, stop requests, IMU validity, and the maneuver/overall run timeouts. If these checks pass, the method calls `_turn_back()`. This method acquires new line sensor readings through `_read_pattern()`, updates the estimated lateral displacement using shared wheel velocities and IMU heading, and checks for a line after the confirmed white interval. If a line qualifies, the task enters `S5_FOLLOW_NEW`, computes the centroid from the existing readings, and publishes line-following wheel requests. If not, it continues producing wheel requests based on the reversed yaw-rate reference. Data is recorded when the logging interval has elapsed.

Calls that may wait: `_read_pattern()` calls `sensor.read()`, which calls `read_raw()`. Its `ADC.read_timed_multi()` call blocks until the fixed sample buffers are filled. Averaging and subsequent calculations also execute before the task yields. IMU values are read from shared data, so this task does not perform an I^2C transaction.

How the next invocation resumes: The helper methods return to `run()`, which reaches `yield self.state`. The next scheduled invocation continues using the retained state, timestamps, and displacement estimate.

My concern is that, although the execution of `TaskLine.run()` during `S4_TURN_BACK` is bounded per invocation, our latest printout testing for IMU data acquisition and averaging gave us a maximum performance time of 18.7 ms—nearly the entire 20 ms period of the Line task. Other tasks cannot execute until this one yields, so the amount of time necessary to read and average the IMU data samples must be split up in some way. This is a diagnosis from our latest source code, since we have not yet updated our line task diagram with the revisions of Lab 0x07.

Improvement Proposal

I propose requiring line detection across consecutive sensor frames before switching from turn-back to line following in `S4_TURN_BACK` of the Line task. The current code can complete successful runs, but a single qualifying frame can also trigger a premature handoff. This change would improve robustness against false detections of a line. I would compare repeated runs with identical settings, checking whether confirmation reduces false handoffs without delaying detection enough to miss the target line.

Development Workflow

The dynamic between myself and my lab partner is communicative and largely effective. We often discuss how to balance the workload of this lab and where we can pick up on each other's code in a non-backtracking manner. Still, a source of unnecessary effort is the time spent verifying that the saved group files match the firmware currently running on Romi. We can reduce this by maintaining one clearly identified and tested version containing the source code and matching configuration/calibration files outside of the lab-specific subfolders. Recording each change and its test result (in text files, for example) would make it easier for either partner to reproduce behavior and build on the other's work.